Longest Common Subsequence

Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, “ABC”, “ABG”, “BDF”, “AEG”, ‘”ACEFG”, .. etc are subsequences of “ABCDEFG”.

Examples:

LCS for input sequences “ABCDGH” and “AEDFHR” is “ADH” of length 3. LCS for input Sequences “AGGTAB” and “GXTXAYB” is “GTAB” of length 4.

Naïve Approach:

The naive solution for this problem is to generate all subsequences of both given sequences and find the longest matching subsequence.

String A.

String B.

Algo:

Find all subsequences of A – $O(2^m)$

For each subsequence check whether it is a subsequence of B – $O(n)$

Find the longest such subsequence.

Time Complexity: Notice that for each character in a string we can either include that character or exclude that character. So every character has 2 choices. If the string has *m* characters then there can be 2^m subsequences. Hence overall time complexity – $O(n2^m)$

Dynamic Programming Approach

Let’s see if this problem possess both the properties of DP.

1. **Optimal Substructure**

String X : $X_1, X_2, X_3 \ldots\ldots\ldots\ldots X_N$

String Y : $Y_1, Y_2, Y_3 \ldots\ldots\ldots\ldots Y_N$

There can be three cases:

1. If $\mathbf{X_N} = = \mathbf{Y_N}$ then we found 1 character that is going to be in the final answer. Then find the LCS in the remaining string:

   LCS(X[0…..N] , Y[0….N]) = LCS (X[0…..N-1], Y[0…..N-1]) + X_N

   = LCS (X[0…..N-1], Y[0…..N-1]) + 1

2. If $\mathbf{X_N}$ **!=** $\mathbf{Y_N}$ then we either remove XN from X and find the LCS (X[0….N-1] , Y[0…..N]) or find the LCS (X[0….N] , Y[0…..N-1]). Since we are interested in longest, so we take the maximum of them.

   LCS(X[0…..N] , Y[0….N]) = max (LCS (X[0…..N-1], Y[0…..N) or
   LCS (X[0…..N], Y[0…..N-1])

3. If any one of the string is empty then LCS is 0.

$$LCS\left(X_i, Y_j\right) = \begin{cases} \emptyset & \text{if } i = 0 \text{ or } j = 0 \\ LCS\left(X_{i-1}, Y_{j-1}\right) \ +1 & \text{if } x_i = y_j \\ \text{longest}\left(LCS\left(X_i, Y_{j-1}\right), LCS\left(X_{i-1}, Y_j\right)\right) & \text{if } x_i \neq y_j \end{cases}$$

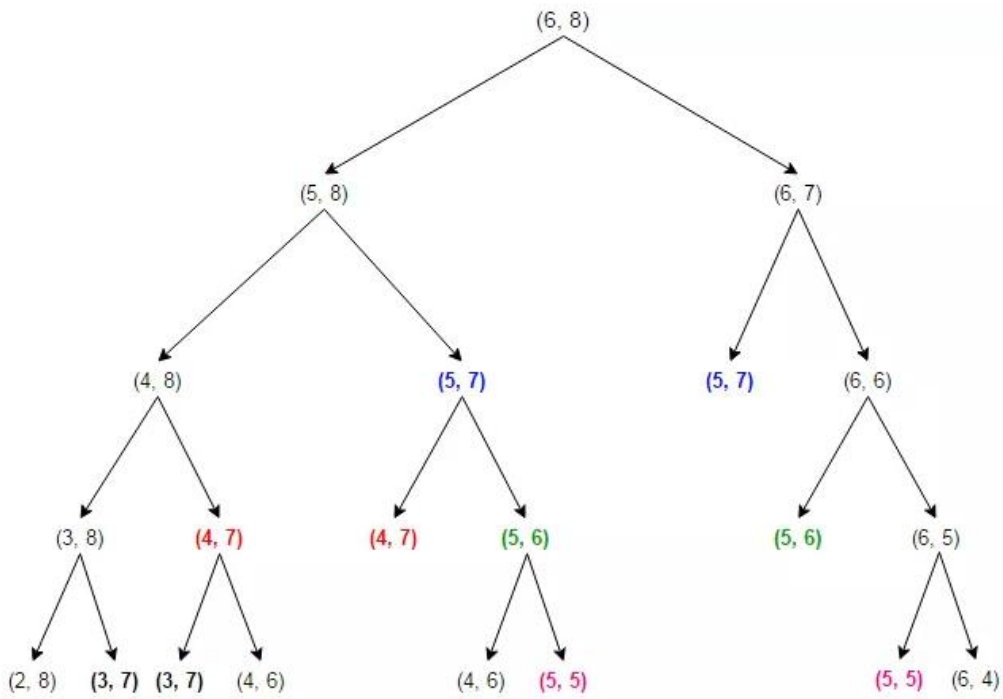
```
int LCSUtil(string X, int M, string Y, int N){
    if(M==0 || N==0)
        return 0;
    if(X[M-1] == Y[N-1])
        return 1 + LCSUtil(X, M-1, Y, N-1);
    else
        return max( LCSUtil(X, M-1, Y, N) , LCSUtil(X, M, Y, N-1));
}

int LCS(string X, string Y){
    return LCSUtil(X, X.length(), Y, Y.length());
}
```

Time complexity of the above naive recursive approach is $O(2^N)$in worst case and worst case happens when all characters of X and Y mismatch i.e., length of LCS is 0.

2. Overlapping Subproblems

The LCS problem exhibits overlapping subproblems. A problem is said to have overlapping subproblems if the recursive algorithm for the problem solves the same subproblem over and over rather than always generating new subproblems.

Let us consider recursion tree for two sequences of length 6 and 8 whose LCS is `0`.



As we can see, the same sub-problems (highlighted in same color) are getting computed again and again. We know that problems having optimal substructure and overlapping subproblems can be solved by dynamic programming, in which subproblem solutions are memoized rather than computed again and again.

Below memoized version follows the top-down approach, since we first break the problem into subproblems and then calculate and store values. **The time complexity of below solution is O(mn) and auxiliary space used by the program is O(mn).**

```
int LCSLength(string X, string Y, int M, int N, unordered_map<string, int>
&lookup){
    // return if we have reached the end of either string
    if (M == 0 || N == 0)
        return 0;

    // construct a unique map key from dynamic elements of the input
    string key = to_string(M) + "|" + to_string(N);

    // if sub-problem is seen for the first time, solve it and
    // store its result in a map
    if (lookup.find(key) == lookup.end()){
        // if last character of X and Y matches
        if (X[M - 1] == Y[N - 1])
            lookup[key] = LCSLength(X, Y, M - 1, N - 1, lookup) + 1;
        else
        // else if last character of X and Y don't match
        lookup[key] = max(LCSLength(X, Y, M, N - 1, lookup),
                          LCSLength(X, Y, M - 1, N, lookup));
    }
    // return the subproblem solution from the map
    return lookup[key];
}

int LCS1(string X, string Y){
    unordered_map<string, int> lookup;
    return LCSLength(X, Y, X.length(), Y.length(), lookup);
}

int main(){
    string X = "AGGTAB";
    string Y = "GXTXAYB";
    cout<<LCS1(X, Y);
}
```

We can also solve this problem in bottom-up manner. In the bottom-up approach, we calculate the smaller values of `LCS(i, j)` first, then build larger values from them.

```
             | 0                                        if i == 0 or j == 0
LCS[i][j] = | LCS[i - 1][j - 1] + 1                    if X[i-1] == Y[j-1]
             | longest(LCS[i - 1][j], LCS[i][j - 1])    if X[i-1] != Y[j-1]
```

Let X be “XMJYAUZ” and Y be “MZJAWXU”. The longest common subsequence between X and Y is “MJAU”. The table below is generated by the function `LCSLength`, shows the lengths of the longest common subsequences between prefixes of X and Y. The ith row and jth column shows the length of the LCS of substring `X[0..i-1]` and `Y[0..j-1]`.

j

i

		0	1	2	3	4	5	6	7
		Ø	M	Z	J	A	W	X	U
0	Ø	0	0	0	0	0	0	0	0
1	X	0	0	0	0	0	0	1	1
2	M	0	1	1	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3	3	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

```
int LCSLength(string X, string Y){

    int M = X.length(), n = Y.length();

    // lookup table stores solution to already computed sub-problems
    // i.e. lookup[i][j] stores the length of LCS of substring
    // X[0..i-1] and Y[0..j-1]
    int lookup[M + 1][N + 1];

    // first column of the lookup table will be all 0
    for (int i = 0; i <= M; i++)
        lookup[i][0] = 0;

    // first row of the lookup table will be all 0
    for (int j = 0; j <= N; j++)
        lookup[0][j] = 0;
```

```
    // fill the lookup table in bottomup manner
    for (int i = 1; i <= M; i++){
        for (int j = 1; j <= N; j++){

            // if current character of X and Y matches
            if (X[i - 1] == Y[j - 1])
                lookup[i][j] = lookup[i - 1][j - 1] + 1;
            // else if current character of X and Y don't match
            else
                lookup[i][j] = max(lookup[i - 1][j], lookup[i][j - 1]);
        }
    }

    // LCS will be last entry in the lookup table
    return lookup[M][N];
}
```

The time complexity of above solution is O(mn) and auxiliary space used by the program is O(mn).

Printing the LCS: XMJYAUZ MZJAWXU

		0	1	2	3	4	5	6	7
		Ø	M	Z	J	A	W	X	U
0	Ø	0	0	0	0	0	0	0	0
1	X	0	0	0	0	0	0	1	1
2	M	0	1	1	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3	3	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

1) Construct L[m+1][n+1] using the steps discussed above.

2) The value L[m][n] contains length of LCS. Create a character array lcs[] of length equal to the length of lcs plus 1 (one extra to store \0).

2) Traverse the 2D array starting from L[m][n]. Do following for every cell L[i][j]

a) If characters (in X and Y) corresponding to L[i][j] are same (Or X[i-1] == Y[j-1]),

then

include this character as part of LCS.

b) Else compare values of L[i-1][j] and L[i][j-1] and go in direction of greater value.

In the above figure, 2 given strings: **XMJYAUZ and MZJAWXU,** the LCS is MJAU.

```
string LCS(string X, string Y){

    string ans;
    if(X.length()==0 || Y.length()==0)
        return ans;

    int M = X.length();
    int N = Y.length();

    int lcs[M+1][N+1]={0};

    for(int i=0; i<=M; i++)
        lcs[i][0] = 0;

    for(int j=0; j<=N; j++)
        lcs[0][j] = 0;

    for(int i=1; i<=M; i++){
        for(int j=1; j<=N; j++){
            if(i==0 || j==0)
                lcs[i][j] = 0;
            else if(X[i-1] == Y[j-1])
                lcs[i][j] = lcs[i-1][j-1] + 1;
            else
                lcs[i][j] = max(lcs[i-1][j], lcs[i][j-1]);
        }
    }
```

```
    int index = lcs[M][N];
    int i=M, j=N;
    while(i>0 && j>0){
        if(X[i-1] == Y[j-1]){
            ans.push_back(X[i-1]);
            i--;
            j--;
        }else if(lcs[i-1][j] > lcs[i][j-1])
            i--;
        else
            j--;
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

int main(){
    string X = "XMJYAUZ";
    string Y = "MZJAWXU";

    string lcs = LCS(X, Y);
    if(lcs.length()!=0)
        cout<<lcs;
}
```